

Multi-Agent Systems and Agentic AI

Practical Examination Report

Frederick Lawrence

CRSid: fl482

Team members for Part I: Baron Gracias, Harvey Bermingham

9th June 2026

Repository and logs. The code used for Part I is available at [GitLab repository](#). Training and evaluation logs are available at [W&B run](#). The adaptive-planning implementation studied in Part II is linked at [cmbagent_1g](#). Replace the Part I placeholders with exact clickable URLs.

Notes on jointly owned work. Sections I.1–I.3 describe experiments run using the shared team TPU and shared repository. The analysis and prose in this report are my own. Section I.4 and Part II are completed individually.

Contents

Part I: GRPO finetuning of Gemma 3 on a TPU

I.1 Reproducing the baseline

Table 1: Baseline run details.

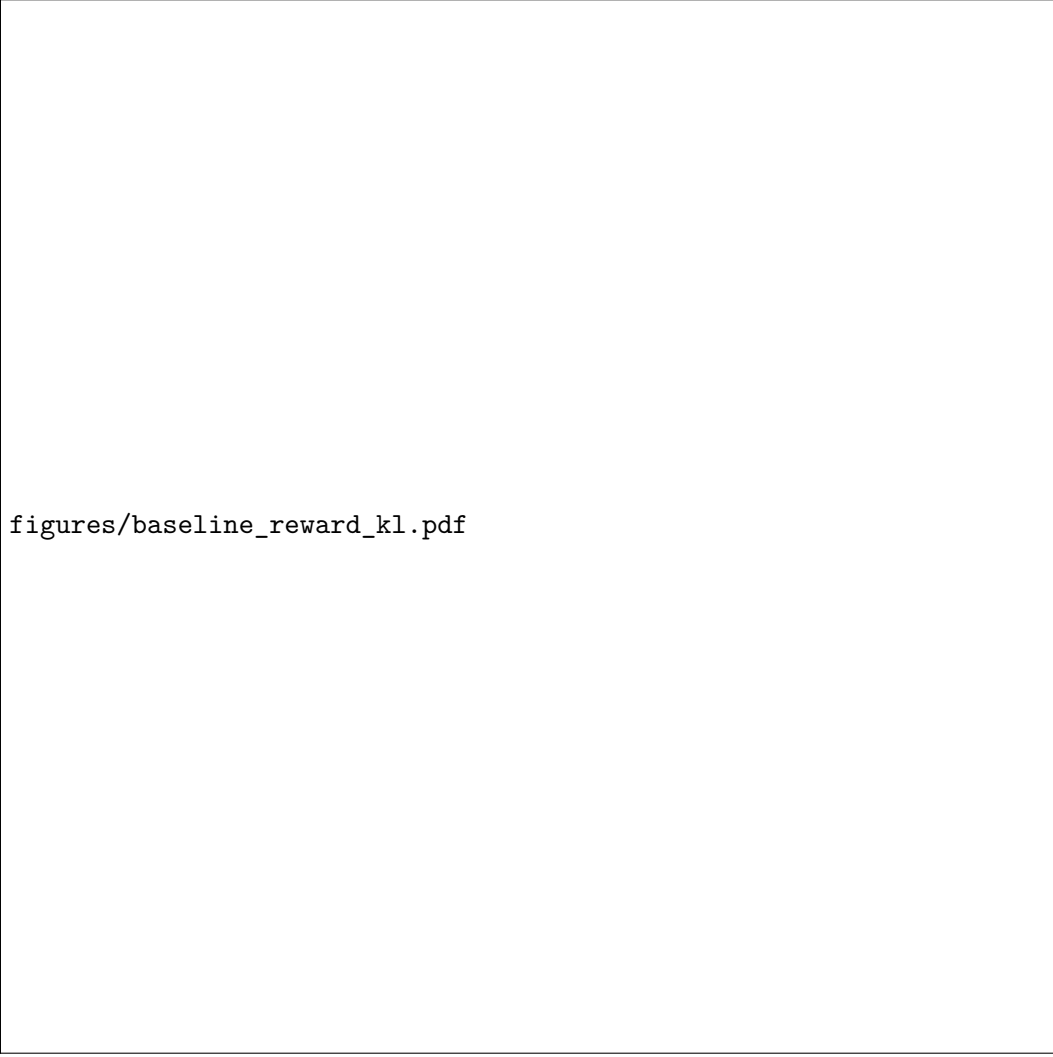
Item	Value
Repository commit	TODO
TPU type	v6e-1
Training command	<code>python scripts/train.py</code>
Evaluation command	<code>python scripts/evaluate.py</code>
Wall-clock time	[TODO: e.g. 6 h 20 min]
GRPO steps completed	[TODO: number of steps]
Held-out split	[TODO: e.g. GSM8K test split]
Evaluation seed	[TODO: fixed seed]

Run details.

Baseline patches. [TODO: State whether the baseline ran as shipped. If not, document the minimal fix required, with the affected file and commit. Keep this short.]

Table 2: Baseline GSM8K accuracy.

Model	Split	Seed	Accuracy
Base model, <code>google/gemma-3-1b-it</code>	[TODO: split]	[TODO: seed]	[TODO: value]
LoRA GRPO checkpoint	[TODO: split]	[TODO: seed]	[TODO: value]



figures/baseline_reward_kl.pdf

Figure 1: Baseline training curves showing mean reward $\bar{r}$ and $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$ against GRPO step.

Evaluation.

I.2 Understanding the baseline

Policies in the LoRA setup. [TODO: Explain what plays the role of π_θ , π_{old} and π_{ref} , and identify which carries trainable parameters.]

Group size and advantage estimator. [TODO: State the group size K from `scripts/config.py`. Identify the Tunix advantage estimator and state whether it is standard group-mean or leave-one-out.]

Reward terms. [TODO: Identify shaping rewards and the true correctness reward in `scripts/rewards.py`. Explain what would happen early in training with correctness reward alone in terms of within-group reward variance σ_r .]

Clipped surrogate. [TODO: Locate where the PPO-style clipped surrogate is implemented in Tunix. State what ε corresponds to in `scripts/config.py` and note any deviation from the lecture form, such as per-token rather than per-sequence ratios.]

I.3 Improving on the baseline

Modification and motivation. [TODO: Describe the principled modification. Examples: leave-one-out advantage, sweep K , change β or ε , modify shaping rewards, add length penalty, change LoRA

rank, learning rate, schedule or generation length. Connect the choice to GRPO theory.]

Controlled comparison. [TODO: State how the comparison was controlled: same data, same evaluation split, same total compute or normalised per step. Mention baseline plus at least two further complete runs, or justify fewer.]

Table 3: Controlled comparison on held-out GSM8K. Include uncertainty using a second seed or a bootstrap confidence interval over evaluation prompts.

Model / run	Modification	Seed(s)	Accuracy	Uncertainty
Base model	None	[TODO:]	[TODO:]	[TODO:]
Baseline GRPO	Default config	[TODO:]	[TODO:]	[TODO:]
Variant 1	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Variant 2	[TODO:]	[TODO:]	[TODO:]	[TODO:]

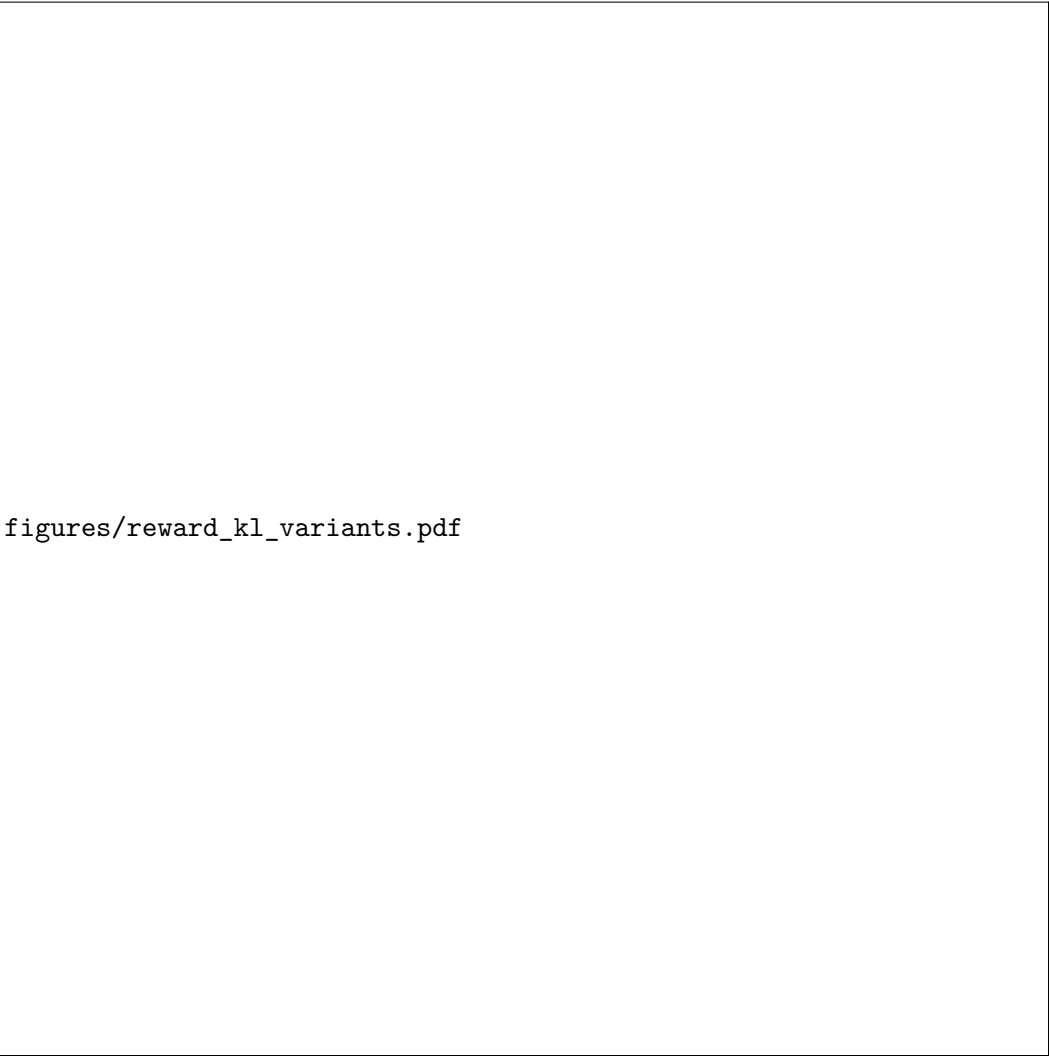


Figure 2: Training curves for the baseline and variants on shared axes, showing mean reward and KL over GRPO step.

figures/grpo_diagnostic.pdf

Figure 3: Diagnostic plot for a known GRPO failure mode, such as advantage distribution, degenerate reward groups, response length, entropy, or KL versus accuracy.

Discussion. [TODO: Connect the empirical findings to lecture theory. State whether the observed behaviour matched the prediction, and diagnose any negative results honestly.]

I.4 GRPO theory

Question 1

(i) Since $\mathbb{E}[\bar{r}] = \mu$,

$$\mathbb{E}[\hat{A}_i] = \frac{\mathbb{E}[r_i] - \mathbb{E}[\bar{r}]}{\sigma} = 0, \quad \mathbb{E}[X_i] = h_i \mathbb{E}[\hat{A}_i] = 0, \quad \mathbb{E}[\hat{g}] = \frac{1}{K} \sum_{i=1}^K \mathbb{E}[X_i] = 0.$$

This expectation averages only reward noise around fixed completions. GRPO is an on-policy method so the completions are sampled from the policy, and the score gradient $\nabla_{\theta} \log \pi_{\theta}(a \mid q)$ can be correlated with the reward of the sampled completion and the policy gradient does not vanish.

(ii) The common random term is the group mean $\bar{r}$, which appears in every advantage:

$$X_i = h_i \hat{A}_i = \frac{h_i}{\sigma} (r_i - \bar{r}).$$

For $i \neq j$,

$$\begin{aligned}\text{Cov}(X_i, X_j) &= \frac{h_i h_j^\top}{\sigma^2} \text{Cov}(r_i - \bar{r}, r_j - \bar{r}), \\ \text{Cov}(r_i - \bar{r}, r_j - \bar{r}) &= 0 - \frac{\sigma^2}{K} - \frac{\sigma^2}{K} + \frac{\sigma^2}{K} = -\frac{\sigma^2}{K}.\end{aligned}$$

Therefore

$$\boxed{\text{Cov}(X_i, X_j) = -\frac{1}{K} h_i h_j^\top, \quad i \neq j.}$$

The negative sign is the result of using a group centred contrast: if one completion is above the group mean, the others are pushed below that same mean. This removes prompt-level reward offsets in GRPO, but the K rollouts are no longer independent samples.

(iii) The naive i.i.d. calculation would instead treat the X_i as independent copies of X_1 , giving

$$\text{Var}_{\text{iid}}(\hat{g}) = \frac{1}{K} \text{Var}(X_1) = \frac{K-1}{K^2} h_1 h_1^\top.$$

The discrepancy is that the true variance depends on the empirical spread of the score vectors around their group mean:

$$\text{Var}(\hat{g}) = \frac{1}{K^2} \sum_i (h_i - \bar{h})(h_i - \bar{h})^\top.$$

Thus the effective sample size should be defined by matching the variance of the actual estimator to an i.i.d. estimator. In any direction u ,

$$u^\top \text{Var}(\hat{g}) u = \frac{1}{K_{\text{eff}}(u)} u^\top \text{Var}(X_1) u,$$

so

$$\boxed{K_{\text{eff}}(u) = \frac{u^\top \text{Var}(X_1) u}{u^\top \text{Var}(\hat{g}) u} = \frac{\frac{K-1}{K} (u^\top h_1)^2}{\frac{1}{K^2} \sum_i \{u^\top (h_i - \bar{h})\}^2}.$$

For diverse score directions, the denominator is typically $O(1/K)$, so K_{eff} grows roughly linearly with K . If all h_i are identical, then $\sum_i \hat{A}_i = 0$ makes $\hat{g} = 0$ exactly; more generally, if all h_i are aligned, only variation in their magnitudes contributes, so extra rollouts that produce the same update direction add little useful RL signal despite increasing sampling cost.

Question 2

(i) Assuming $\pi_t(a) > 0$ is on its support and with finite normalisation constant, any optimiser must have support contained in that of π_t otherwise $\text{KL}(\pi \| \pi_t) = \infty$. With λ enforcing $\sum_a \pi(a) = 1$, stationarity gives

$$A_t(a) - \frac{1}{\eta} \left(\log \frac{\pi(a)}{\pi_t(a)} + 1 \right) + \lambda = 0.$$

Normalising the resulting exponential tilt yields the unique optimiser

$$\pi_{t+1}(a) = \frac{\pi_t(a) \exp\{\eta A_t(a)\}}{Z_t}, \quad Z_t = \sum_b \pi_t(b) \exp\{\eta A_t(b)\}. \quad (1)$$

The objective is strictly concave on this support, so the optimiser is unique. This is the KL mirror-descent or exponential-tilting update: small η gives a conservative RLHF step near π_t , while large η concentrates probability on high-advantage responses but cannot put mass on responses assigned zero probability by π_t .

(ii) Let $F_t(\pi) = \mathbb{E}_{a \sim \pi}[A^{\pi_t}(a)] - \eta^{-1} \text{KL}(\pi \| \pi_t)$. The defining property of the true advantage is $\mathbb{E}_{a \sim \pi_t}[A^{\pi_t}(a)] = 0$, so $F_t(\pi_t) = 0$. Optimality of ?? then implies

$$J(\pi_{t+1}) = \mathbb{E}_{\pi_{t+1}}[A^{\pi_t}] \geq \frac{1}{\eta} \text{KL}(\pi_{t+1} \| \pi_t) \geq 0 = J(\pi_t).$$

Thus the exact unrestricted update is monotone in this advantage objective. In neural-network GRPO the exponential-tilted distribution may be outside the model or LoRA-adapter family, and the advantage is a finite-sample, group-normalised estimate. Either approximation, especially with degenerate groups where σ_r is tiny or zero, can break the guarantee.

(iii) For $\hat{A}_t > 0$, clipping removes the incentive to increase the sampled ratio beyond $1 + \varepsilon$; for $\hat{A}_t < 0$, it removes the incentive to decrease it below $1 - \varepsilon$. The resulting trust region is advantage-dependent, samplewise, and soft: it clips the objective on sampled actions, but does not enforce the ratio bounds after a shared neural update. By contrast, $\text{KL}(\pi_\theta(\cdot | q) \| \pi_{\text{old}}(\cdot | q)) \leq \delta$ is one hard, distribution-level constraint; it can permit a large change on some actions provided the joint KL budget remains satisfied. In LLM RLHF this means PPO clipping can miss unsampled completions and prompt-level KL spikes, whereas a KL constraint directly budgets total distributional drift.

Question 3

(i) For any integrand f where $\mathbb{E}_{\pi_{\text{old}}}[f] = \mathbb{E}_{\pi_{\text{mix}}}[wf]$, the importance weighing w_i is

$$w_i = \frac{\pi_{\text{old}}(a_i | q)}{\pi_{\text{mix}}(a_i | q)} = \frac{\pi_{\text{old}}(a_i | q)}{(1 - \alpha)\pi_{\text{old}}(a_i | q) + \alpha\pi_{\text{unif}}(a_i | q)}. \quad (2)$$

Therefore, the importance-corrected gradient estimator $\hat{g}_{\text{mix}}$ is

$$\hat{g}_{\text{mix}} = \frac{1}{K} \sum_{i=1}^K w_i \nabla_\theta \log \pi_\theta(a_i | q) \frac{r_i - \bar{r}}{\sigma_r}, \quad a_i \stackrel{\text{iid}}{\sim} \pi_{\text{mix}}. \quad (3)$$

(ii) For each fixed realised action, $\pi_{\text{mix}}(a_i | q) \rightarrow \pi_{\text{old}}(a_i | q)$ and hence $w_i \rightarrow 1$. The realised r_i , $\bar{r}$, and σ_r are unchanged, so ?? tends to $\hat{g}_{\text{GRPO}}$.

(iii) Linearity of expectation gives

$$V^{\pi_{\text{mix}}}(q) = (1 - \alpha)V^{\pi_{\text{old}}}(q) + \alpha V^{\pi_{\text{unif}}}(q).$$

Consequently, multiplying by w_i outside the advantage does not change the fact that the unweighted $\bar{r}$ estimates $V^{\pi_{\text{mix}}}$ rather than $V^{\pi_{\text{old}}}$. For the fixed-scale target $\mathbb{E}_{\pi_{\text{old}}}[\nabla_\theta \log \pi_\theta(a | q)(R(q, a)V^{\pi_{\text{old}}})/s]$ the leave-one-out (LOO) weighted baseline is needed.

$$\hat{V}_{-i} = \frac{1}{K-1} \sum_{j \neq i} w_j r_j, \quad \hat{g}_{\text{mix}}^{\text{LOO}} = \frac{1}{K} \sum_i w_i \nabla_\theta \log \pi_\theta(a_i | q) \frac{r_i - \hat{V}_{-i}}{s}. \quad (4)$$

$\mathbb{E}_{\pi_{\text{mix}}}[\hat{V}_{-i}] = V^{\pi_{\text{old}}}$ and $\hat{V}_{-i}$ is independent of a_i , so ?? is unbiased for that old-policy score-advantage expectation. The common self-normalised baseline $\hat{V}_{\text{SN}} = \sum_j w_j r_j / \sum_j w_j$ is often lower variance and consistent, but it is biased at finite K ; using the same group's empirical σ_r adds the usual GRPO normalisation bias and fails when the group reward variance is zero. Thus the reweighted LOO advantage is

$$\tilde{A}_i^{\text{LOO}} = \frac{r_i - \hat{V}_{-i}}{s}.$$

(iv) There are two variance effects. First, the unweighted reward variance under the proposal is

$$\begin{aligned}\text{Var}_{\pi_{\text{mix}}}(R) &= (1 - \alpha) \text{Var}_{\pi_{\text{old}}}(R) + \alpha \text{Var}_{\pi_{\text{unif}}}(R) \\ &\quad + \alpha(1 - \alpha)(V^{\pi_{\text{old}}} - V^{\pi_{\text{unif}}})^2.\end{aligned}$$

Thus mixing increases advantage variance when the uniform sampler has much larger reward variance or a very different mean, as with rare correct answers mixed with many invalid LLM completions. It decreases raw advantage variance when $V^{\pi_{\text{unif}}} \simeq V^{\pi_{\text{old}}}$ and $\text{Var}_{\pi_{\text{unif}}}(R) < \text{Var}_{\pi_{\text{old}}}(R)$, for example if uniform exploration smooths a spiky old-policy reward distribution.

Second, importance weighting changes the gradient estimator variance. For a scalar projection Y of the score–advantage term,

$$\mathbb{E}_{\pi_{\text{mix}}}[w^2 Y^2] = \mathbb{E}_{\pi_{\text{old}}}[w Y^2].$$

Therefore lower raw reward variance does not guarantee lower gradient variance: large- $|Y|$ samples in regions where $\pi_{\text{old}} > \pi_{\text{mix}}$ get $w > 1$ and can dominate, while adding proposal mass in regions that contribute substantially to the old-policy second moment can reduce variance by lowering their importance weights.

(v) Under the stated assumption,

$$w_i = c^T = \exp(T \log c) \longrightarrow 0 \quad \text{exponentially as } T \rightarrow \infty.$$

When $\alpha > 0$, mixture samples include tokens to which the uniform component assigns more probability than π_{old} ; more formally, $\mathbb{E}_{\pi_{\text{mix}}}[\log(\pi_{\text{old}}/\pi_{\text{mix}})] = -\text{KL}(\pi_{\text{mix}}\|\pi_{\text{old}}) < 0$, so the geometrically typical token ratio is below one. Consequently most long responses have negligible weight while a few rare trajectories dominate, which is disastrous for long-form LLM RLHF because the effective sample size collapses exponentially with response length. One concrete fix is to scale the mixture coefficient as $\alpha = \gamma/T$, keeping the cumulative proposal perturbation, and hence typical log-weight magnitude, $O(1)$; the trade-off is weaker uniform exploration for longer responses.

Part II: Adaptive planning in cmbagent_1g

II.1 Workflow diagram

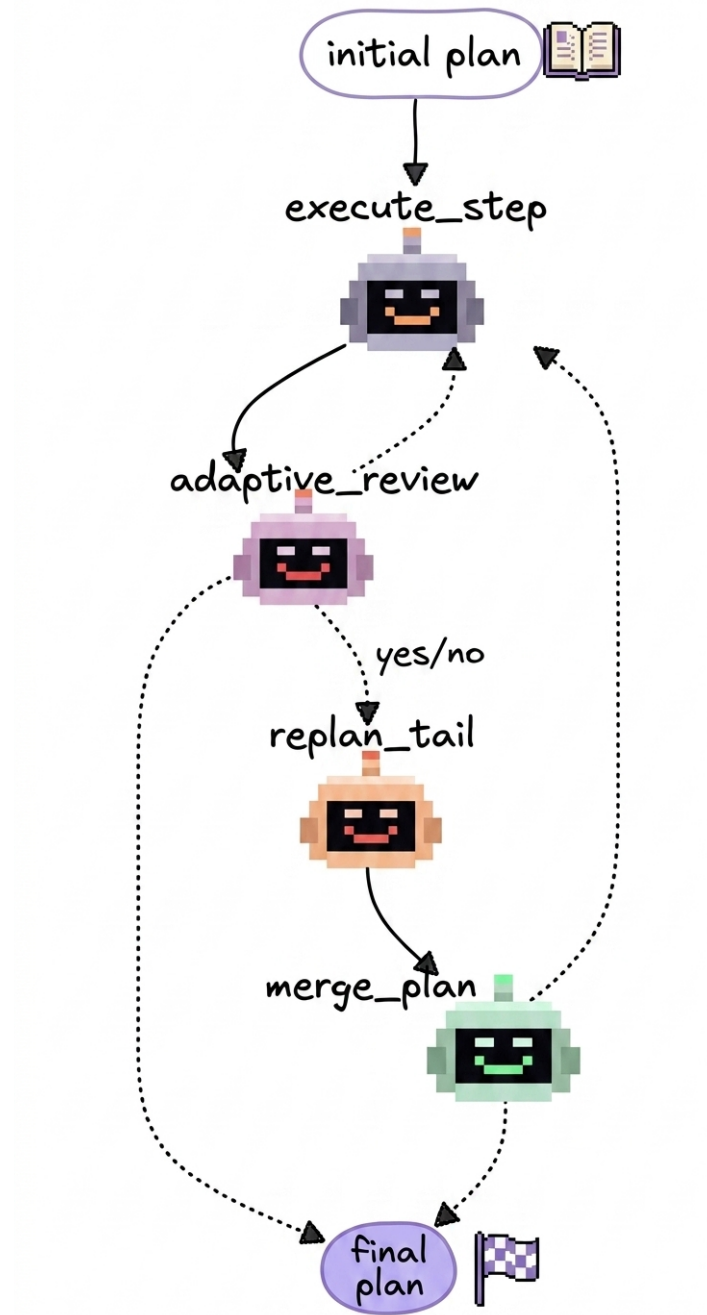


Figure 4: Workflow diagram of the adaptive-planning graph in `cmbagent_1g`. Dotted edges are conditional routes from the reviewer and merge routers; solid edges are unconditional graph transitions.

Walkthrough. I studied `cmbagent_1g`. The initial planning graph is open-loop: a planner and plan reviewer run a bounded propose–critique loop and produce a structured `Plan`. The adaptive graph consumes that plan. Its `execute_step` node is the existing `deep_research.run_step`: it runs the current `Step`, appends a summary and outcome, checkpoints the deep-research run, and increments the 1-based `step_index`. After every executed step, `adaptive_review` asks a critic model whether the remaining steps should be rewritten in light of the latest step summary and outcome. The revision trigger is therefore the structured field `AdaptiveReview.needs_adaptation`. If it is true, `replan_tail`

asks the planner model for a replacement tail and `merge_plan` splices `completed_prefix + new_tail` back into `plan`. The completed prefix, its summaries, outcomes, and workspace artefacts are held fixed; only not-yet-executed steps may change. The graph terminates when the latest step is unfulfilled, when `step_index` is beyond the current plan length, or when a rewrite leaves no remaining steps. Otherwise control returns to `execute_step`.

II.2 Problems

Problem 1: an informal feedback controller. The adaptive graph is closed-loop, but the feedback controller is weak. As shown in ??, the only trigger for replanning is the LLM-generated field `AdaptiveReview.needs_adaptation`, computed from the latest step summary, latest outcome, and remaining tail. There is no explicit uncertainty estimate, cost model, value-of-information test, or task-level error signal. This means the system can observe execution feedback but cannot reliably decide whether rewriting the plan is worth the extra cost. In practice, this can cause unnecessary replanning after harmless new information, or leave an invalid tail unchanged when the reviewer fails to recognise that the plan has become stale. [TODO: manually review]

Problem 2: failures end the outer loop rather than trigger repair. If the code returns an error or fails to execute properly, the `adaptive_review` agent will mark the step as unfulfilled and the loop will immediately end. During execution, the engineer agent can invoke the `self_debug` loop but this targets bugs in the code rather than fundamental flaws in the plan. This is an issue because code failing to run is often one of the clearest pieces of evidence that the plan itself is wrong. It does not give the system the chance to review the error and rewrite dependent future steps. The result is a system that can adapt after successful steps, but is least adaptive when plan level recovery would be most useful.

Problem 3: tail rewrites are myopic and unvalidated. The initial planner uses a reviewer loop and keeps review history, whereas the adaptive replanner directly produces a new tail. The reviewer sees the latest result and the remaining steps, but not the full initial planning transcript or a rich history of earlier adaptations. After `replan_tail`, `merge_plan` splices the new tail into the plan without checking task coverage, dependencies on completed artefacts, plan length, or whether the recommendations were actually followed. The dynamic schema prevents invalid agent names, but it does not ensure that the repaired plan is better. This makes the adaptive update potentially destabilising: a weak rewrite can replace a sound remaining plan. The risk is increased by the lack of a revision budget and by the thin `adaptive_history`, which omits the old tail, new tail, and reviewer reason from the persisted evidence. [TODO: manually review - no check that the tail is actually better]

II.3 Proposed improvements

Structured adaptation policy. Extend `AdaptiveReview` with structured fields for reason category, confidence, expected benefit, extra steps, and estimated cost. Route to `replan_tail` only when an explicit policy is met, for example a high-confidence hard invalidation or expected benefit above a threshold. This makes the controller auditable and reduces gratuitous replanning. The trade-off is prompt and schema complexity, and the numerical scores would still be model judgements rather than calibrated measurements.

Failure-aware replanning path. Before ending on `fulfilled=False`, add a bounded failure-review route that gives the replanner the failed step, failure reason, completed summaries, and remaining tail. It could insert a repair step, replace the failed step, or mark the failure unrecoverable. This separates local retry from plan-level repair and better matches closed-loop planning, where failures are observations to condition on. The trade-off is extra latency after failures and the danger of repeatedly trying to repair an impossible task, so it needs a small retry/revision budget.

Validated, budgeted tail replacement. [TODO: `merge_plan` should validate the plan before merging (effectively same as adaptive review)] Insert a `validate_new_tail` node before `merge_plan`. It should check that the new tail preserves global task coverage, uses only available agents, does not duplicate completed steps, respects a maximum plan length, and follows the reviewer's recommendations; invalid tails can be sent back to `replan_tail` with feedback, bounded by a `PlanContext` revision budget. Persist each revision with old tail, new tail, reviewer reason, model names, and timestamps. This improves stability and auditability, at the cost of another critic call per adaptation, larger checkpoints, and some risk of rejecting a creative but useful repair.

References

- Hu, E. J. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. <https://arxiv.org/abs/2106.09685>
- Tunix documentation or source link used for GRPO implementation: <https://github.com/google/tunix>
- Exact cmbagent_lg adaptive-planning commit studied: https://github.com/borisbolliet/cmbagent_lg/tree/d7d0592a714e4cc01c97f1b77afdd57a208b18db

A Appendix

A.1 Additional run details

[TODO: Optional non-essential logs, extra tables, hyperparameter dumps, and supplementary figures.]